

# JARVIS-LOCAL — Execution Handbook v1.1

**Changelog v1.0 → v1.1:** Added the Desktop Avatar subsystem (companion character on screen: state-reactive animations, cursor-tracking eyes, click interactions, hot-swappable avatar packs). Touched sections: §1, §5, §6, §7, §9, §10, §13-adjacent (new §13A), §15, §17, §19, §20, §22, §23, §24, §25, §26, and new Appendix B (avatar pack authoring). Version bump per §29 (interface additions in §10).

**Purpose of this document.** This is the project's source of truth. Any model or engineer continuing this project must read this document first and treat it as binding unless a Decision Log entry supersedes it. It encodes not just what to build but why each decision was made, so future work reproduces the same engineering judgment. Where the original owner never settled a choice, a ⚠ DECISION REQUIRED block exists — do not silently resolve these; surface them to the owner.

---

## 1. Vision

A fully local, privacy-first, Jarvis-style voice assistant running on the owner's personal laptop. It listens for a wake word, transcribes speech (Indian English, including Indian names), reasons with a local LLM, optionally looks at the screen, executes safe system actions (shell/apps/files), and speaks responses with low perceived latency via end-to-end streaming.

**New in v1.1:** the assistant has a visible presence — a small animated avatar (a cat, an axolotl, or any pack the owner installs) that sits on the desktop, reflects the assistant's state (idle / listening / thinking / speaking / error), tracks the mouse cursor with its eyes, and can be clicked to activate listening without the wake word. The avatar is a view of the assistant, never a second brain: it renders state, it never owns it.

What this project is NOT: not a cloud service, not multi-user, not a product to ship, not an autonomous agent that acts without oversight. It is a single-owner personal tool where privacy and responsiveness outrank raw intelligence. The avatar likewise is a cosmetic/ergonomic layer — the assistant must remain fully functional with the avatar disabled (`(avatar.enabled: false)`).

## 2. Design Philosophy

1. **Perceived latency > actual latency.** The user's experience is "end of my speech → first spoken word." Every architectural choice (streaming, sentence chunking, concurrent tracks) serves this metric, not total completion time. *Avatar corollary:* the avatar makes state visible — seeing "listening" ears perk up within 100 ms is itself a perceived-latency win, even before any audio plays.
2. **Local-first, always.** No audio, screenshots, or transcripts leave the machine. Cursor position (used for avatar eye-tracking) is likewise never logged or transmitted; it is consumed in-memory by the renderer and discarded.
3. **Single process until proven otherwise.** In-memory `asyncio.Queue`s beat any IPC for a laptop app. The avatar renderer joins the same process via a Qt/asyncio shared event loop (§7) rather than spawning a helper process. Services/ZeroMQ are added only when a trigger in §27 fires.
4. **The model proposes; the code disposes.** The LLM never has raw shell. Unchanged. The avatar has *no* channel to the LLM or the tool executor; it only consumes bus events and emits one user-intent event (click-to-listen).
5. **Degrade gracefully across hardware.** The same architecture runs on CPU-only or 12GB+ GPU; only model selection changes (§8). The avatar must run within a small fixed CPU budget on Tier A (§6, §22) — sprite-sheet 2D animation, no GPU compositing requirements.

### 3. Non-negotiable Principles

P1 — **No raw shell for the model.** All actions via the tool registry. P2 — **Whitelist, never blacklist.** P3 — **Destructive actions require confirmation against the specific resolved command.** P4 — **Non-privileged execution.** P5 — **Every long await is cancellable.** Applies to the avatar's animation timers too: shutting down or disabling the avatar must never hang the loop. P6 — **Audio callbacks do queue-puts only.** P7 — **Append-only audit log for every executed command.** P8 — **Screen capture is on-demand only, never continuous.** P9 (*new*) — **The avatar is a passive view plus one input affordance.** It may (a) render state from `STATE_CHANGED` events and (b) emit `AVATAR_ACTIVATE` on click. It must never mutate orchestrator state directly, call tools, read transcripts, or capture the screen. Why: the render layer is the least security-reviewed code in the system (third-party art packs, GUI framework); keeping it write-blind to everything except one intent event means a compromised or buggy avatar pack cannot escalate. P10 (*new*) — **Avatar packs are data, never code.** A pack is images + a declarative manifest. No Python, no scripts, no eval of any manifest field. Why: packs will be downloaded/shared casually; executable packs would be a trivial malware vector on a machine that runs a tool executor.

### 4. Success Criteria

- SC1: Wake-word → assistant listening: < 300 ms perceived.
- SC2: End-of-speech → first spoken word: < 1.5 s GPU tier, < 3 s CPU tier.
- SC3: Barge-in silences playback in < 100 ms.
- SC4: Zero commands executed outside the whitelist across the audit log's lifetime.
- SC5: Indian names transcribed correctly  $\geq 90\%$  (self-recorded test set, §23).
- SC6: Runs a full day session without memory growth or zombie processes (now includes the render loop: no unbounded pixmap/frame caching).
- SC7: "What's on my screen" useful description in < 5 s (GPU) / < 15 s (CPU).
- SC8 (*new*): Avatar reflects a state transition within 100 ms of the `STATE_CHANGED` event.
- SC9 (*new*): Avatar idle rendering (eye tracking + idle animation) uses < 3% of one CPU core and < 80 MB RSS attributable to the render layer.
- SC10 (*new*): Switching avatar packs at runtime (via config-designated hotkey/tray menu or voice) completes in < 1 s with no assistant downtime.

## 5. Functional Requirements

- FR1 Wake-word activation (`openWakeWord`), always-on, CPU-resident.
- FR2 Streaming STT with partial results; Indian English support; contact-name biasing.
- FR3 Conversational responses from a local LLM with rolling context.
- FR4 Tool calling: `open_app`, `list_dir`, `check_disk`, `run_command` (whitelisted), extensible registry.
- FR5 On-demand screen understanding: mss screenshot → VLM → description.
- FR6 Streaming TTS with sentence-level chunking; playback begins on first sentence.
- FR7 Barge-in: user speech during SPEAKING cancels TTS and playback.
- FR8 Tool results feed back into generation in the same turn.
- FR9 All configuration in one `config.yaml`.
- FR10 (*new*) **Avatar window**: frameless, transparent-background, always-on-top, draggable widget rendering the active avatar pack; position persisted across restarts; optional click-through mode.
- FR11 (*new*) **State-reactive animation**: the avatar plays the animation mapped to the current orchestrator state (`idle`, `listening`, `thinking`, `speaking`, `error`), switching within SC8's budget. Missing animations in a pack fall back per the manifest's fallback chain (Appendix B).
- FR12 (*new*) **Cursor eye-tracking**: the avatar's eye layer (pupils) follows the OS cursor position, clamped to the eye socket radius defined in the pack manifest; smoothed (lerp) so motion looks organic; polled at  $\leq 30$  Hz; disableable in config.
- FR13 (*new*) **Idle micro-behaviors**: random blink and occasional idle fidget animations (if the pack provides them) on a jittered timer, so the avatar feels alive without consuming CPU meaningfully.
- FR14 (*new*) **Click-to-activate**: left-clicking the avatar emits `AVATAR_ACTIVATE`, which the orchestrator treats exactly like a wake-word detection (IDLE → LISTENING). Right-click (or long-press) opens a minimal menu: switch avatar, toggle eye tracking, hide avatar, quit.
- FR15 (*new*) **Avatar pack switching**: multiple packs live in `avatars/`; the active one is selected in config, from the avatar's context menu, or by voice ("switch to the axolotl") via a new non-destructive `set_avatar(name)` tool. Switching hot-reloads the pack without restarting the assistant (the *one* sanctioned exception to frozen config — see §15 for why this is safe where whitelist hot-reload is not).
- FR16 (*new*) **Pack extensibility**: adding a new avatar requires only dropping a conforming pack directory into `avatars/` (format in Appendix B). No code changes; the pack appears in the switch list on next scan.

Out of scope for v1: multi-step autonomous GUI control, multi-device access, cross-session memory, Hindi output, and *avatar extras*: speech-synced lip animation beyond a generic "talking" loop, physics/dragging inertia, avatar walking along window edges, multiple simultaneous avatars. All listed in §28.

## 6. Non-functional Requirements

- **NFR1 Privacy:** no network calls except `(localhost:11434)` (Ollama). The avatar layer makes zero network calls; cursor coordinates never leave the render module and are never logged at any level.
- **NFR2 Latency:** budgets per §22.
- **NFR3 Resource ceiling:** assistant idle uses  $< 5\%$  CPU *including* the avatar (avatar's share  $\leq 3\%$  of one core, SC9). Render loop must be event/timer-driven, never a busy loop; animation timers pause when the pack's current animation is a static frame.
- **NFR4 Robustness:** any stage crash is contained. A crash in the avatar layer must never take down audio or the orchestrator: the supervisor restarts the avatar widget; if it crashes  $3\times$  in 60 s it stays disabled for the session with a spoken notice.
- **NFR5 Auditability:** unchanged; `(set_avatar)` executions appear in the app log (not the audit log — it is not a command subprocess).

## 7. System Architecture

Single Python 3.11 process. Concurrency domains, updated:

1. **Audio-in OS thread** — PortAudio callback  $\rightarrow$  thread-safe queue (janus). 16 kHz mono int16, blocksize 512.
2. **Unified GUI + asyncio event loop** — Qt (PySide6) event loop integrated with asyncio via `qasync`, so one loop runs: wake+VAD task, STT task, orchestrator, chunker, TTS worker, tool executor, interrupt bus, *and* the avatar widget's paint/timer events. Why `qasync` over a separate GUI thread or process: Qt demands to own the main thread; running two loops and bridging them reintroduces exactly the IPC/serialization cost DL-1 rejected. `qasync` gives native Qt rendering and native asyncio in one thread with zero marshalling. Risk (Qt paint work starving the pipeline) is bounded because avatar paints are tiny blits at  $\leq 30$  fps, and it is measured by the latency CI (§19). Escape hatch: if CI shows loop starvation, the avatar moves behind trigger (c) of §27 — it is the most naturally severable component precisely because P9 restricts it to two event types.
3. **Audio-out OS thread** — PortAudio callback pulls PCM from thread-safe queue.

Pipeline (happy path) unchanged: mic  $\rightarrow$  ring buffer  $\rightarrow$  wake+VAD  $\rightarrow$  STT  $\rightarrow$  orchestrator  $\rightarrow$  {tool executor || chunker  $\rightarrow$  TTS  $\rightarrow$  playback}. New fan-out: the interrupt bus's `(STATE_CHANGED)` events are additionally consumed by the avatar animator; the avatar publishes `(AVATAR_ACTIVATE)` into the same bus consumed by the orchestrator.

State machine (owned solely by orchestrator): `(IDLE  $\rightarrow$  (wake | AVATAR_ACTIVATE)  $\rightarrow$  LISTENING  $\rightarrow$  (VAD end)  $\rightarrow$  THINKING  $\rightarrow$  (first token)  $\rightarrow$  SPEAKING  $\rightarrow$  (complete|cancel)  $\rightarrow$  IDLE)`; barge-in: `(SPEAKING  $\rightarrow$  LISTENING)`. The avatar renders these states; it never holds its own copy of "what state we're in" beyond the last event received (stateless view — why: two sources of state truth was the race-bug class §7 was designed to kill).

## 8. Hardware Tiering

Unchanged (STT/LLM/VLM/TTS table as v1.0). The avatar is tier-independent: 2D sprite blitting is trivially within budget on Tier A, so there is deliberately **no** tiered avatar behavior (e.g., no "fancier avatar on GPU"). Why: tier must only select models, never fork feature behavior — that rule keeps the tier switch a one-line config change.

⚠ DECISION REQUIRED (D-1): unchanged — confirm deployment tier before Milestone 2. Default: Tier A.

## 9. Component Responsibilities & Module Boundaries

Rule unchanged: a module may import from `bus.py`, `config`, and its own package; cross-package communication goes through §10/§11 queue/event interfaces only.

```
jarvis/
├─ main.py                # composition root: qasync loop, threads+tasks,
supervisor
├─ config.yaml            # single source of runtime config (§15)
├─ bus.py                 # interrupt bus, shared events, state enum
├─ audio/
|   ├─ input_thread.py
|   └─ output_thread.py
├─ perception/
|   ├─ wake_vad.py
|   ├─ stt.py
|   └─ vision.py
├─ brain/
|   ├─ orchestrator.py
|   └─ chunker.py
├─ voice/
|   └─ tts.py
├─ actions/
|   ├─ registry.py        # + set_avatar tool entry
|   └─ executor.py
├─ avatar/                # NEW package – the entire render layer
|   ├─ window.py          # frameless QWidget: transparency, drag, click,
tray/context menu
|   └─ animator.py        # STATE_CHANGED → animation selection; blink/fidget
timers; frame stepping
|   └─ eyes.py            # cursor polling (≤30 Hz), socket clamping, lerp
smoothing
|       └─ packs.py        # pack discovery, manifest parsing+validation,
sprite loading, hot-swap
├─ avatars/               # NEW data dir – one subdirectory per
pack (Appendix B)
|   ├─ cat/
|   └─ axolotl/
└─ tests/                 # mirrors package structure (§23)
```

Exclusive responsibilities, extended: only `(orchestrator.py)` mutates state; only `(executor.py)` spawns command subprocesses; only `(wake_vad.py)` publishes barge-in; **only** `(avatar/window.py)` publishes `(AVATAR_ACTIVATE)`; **only** `(avatar/packs.py)` reads pack files **from disk**. `(avatar/)` imports Qt; no other package may import Qt (why: keeps the whole pipeline headless-testable, M3's property).

## 10. Data Flow & Event Flow

Queue dataclasses (all `(@dataclass(frozen=True, slots=True))`) — v1.0 set unchanged (`(AudioFrame)`, `(PartialTranscript)`, `(FinalTranscript)`, `(Token)`, `(Phrase)`, `(PcmChunk)`, `(ToolCall)`/`(ToolResult)`, `(ScreenQuery)`/`(ScreenDescription)`), plus:

- `(AvatarCommand(action: Literal["set_pack", "toggle_eyes", "hide", "show"], arg: str|None))` — orchestrator/tool-layer → avatar. Carries pack switches and menu-equivalent voice commands.
- `(AvatarStatus(pack: str, visible: bool, error: str|None))` — avatar → orchestrator (informational; lets the assistant *speak* pack-load failures per §16's "user is the operator" rule).

Events (`(bus.py)`):

- `(CANCEL)` — unchanged.
- `(STATE_CHANGED(new_state))` — was informational; now has a real consumer (avatar animator). Its delivery latency is therefore budgeted (SC8) and covered by the latency CI.
- `(AVATAR_ACTIVATE)` (*new*) — click-to-listen intent; consumed only by the orchestrator, valid only in IDLE (ignored otherwise — clicking a speaking avatar is not barge-in; barge-in stays voice-only to keep §25 D-5's analysis intact).

Backpressure: unchanged for audio. Avatar event consumption is fire-and-forget with a depth-1 "latest wins" mailbox — if the animator is mid-frame when a new state arrives, it drops the stale target and jumps to the newest (why: animation must never queue up behind reality; showing the *current* state late beats showing an *old* state on time).

## 11. Interfaces & Contracts

External contract set unchanged (Ollama only). The avatar adds **no external interface** — Qt is a library dependency, not a service.

Internal: the §10 payloads ARE the interfaces; changing a field updates this handbook and all producers/consumers in the same commit.

New internal contract — **the avatar pack manifest** (`(avatars/<name>/avatar.yaml)`, full spec Appendix B). `(packs.py)` validates it against a schema at load; an invalid pack is rejected with a spoken+logged error and the previous pack stays active (never crash the widget on bad data — packs are user-authored).

## 12. State & Memory Architecture

Unchanged. Avatar-relevant note: the only persisted avatar state is `{active_pack, window_x, window_y, eyes_enabled, visible}` written to a small `avatar_state.json` beside the config on clean shutdown (why persisted separately from `config.yaml`: config is frozen-at-startup and human-authored; window position is machine-written ephemera — mixing them would either violate the frozen-config rule or make the config file churn).

⚠ DECISION REQUIRED (D-2): transcripts to disk? Default no. Unchanged.

## 13. Agent, Prompt & Tool Architecture

Unchanged, plus one tool:

- `set_avatar(name: str)` — `destructive: false`, no confirmation, handler publishes `AvatarCommand("set_pack", name)`; validates `name` against the discovered pack list before publishing (unknown pack → spoken "I don't have an avatar called X; I have: cat, axolotl"). Timeout 2 s. Why a tool and not intent-parsing in the orchestrator: keeps *all* actions in the one reviewable registry (DL-5's logic), even cosmetic ones.

System prompt addition (content spec): the assistant knows it has a visible avatar and the names of installed packs (injected from pack discovery at startup, like the contact-name glossary), so "switch to the cat" resolves naturally.

### 13A. Avatar Subsystem Design (*new*)

**Rendering model.** Layered 2D sprite compositing per frame, painted back-to-front:

1. `body` layer — current animation's frame (from a sprite sheet or numbered PNGs).
2. `eyes` layer (*optional*) — a socket-anchored pupil image per eye, offset each tick by the clamped, lerped cursor vector from `eyes.py`. Packs without an eye layer simply skip this composite (a sleeping-cat pack may not want tracking eyes).
3. `overlay` layer (*optional*) — blink frames, emotes (e.g., "!" on error), drawn on their own timers over the body.

Why layered composite instead of pre-baking eye positions into animations: eye tracking is continuous (cursor can be anywhere); baking would need hundreds of frames per animation. Two small pupil blits per tick is the cheap, general solution, and it's what makes eye-tracking a *pack-manifest feature* rather than a per-pack art burden.

**Animation state mapping.** `animator.py` holds a pure function `state → animation_name` using the manifest's `states:` table, with a fallback chain (`speaking → talking → idle`) so minimal packs (idle-only) still work everywhere. Blink/fidget run on jittered timers (blink every 3–8 s, fidget every 20–60 s) only while in `idle`; any state change cancels them (P5 applies).

**Frame stepping.** A single `QTimer` per active animation at the manifest's fps (cap 30). Static single-frame animations stop the timer entirely (NFR3). Frames are decoded once at pack load and cached; cache is dropped on pack switch (SC6).

**Eye tracking.** `QCursor.pos()` polled by a 30 Hz timer (no global input hooks, no OS event taps — polling is privacy-inert and portable); vector from each socket center to cursor, clamped to `socket_radius`, then `pupil_pos += (target - pupil_pos) * 0.25` per tick for organic lag. Cross-platform note: Wayland restricts global cursor position; on Wayland the eyes degrade to tracking only while the cursor is over/near the avatar window, and this limitation is logged once at startup (ties to D-3).

**Window behavior.** Frameless, `WA_TranslucentBackground`, always-on-top, drag-to-move (position saved), left-click = `AVATAR_ACTIVATE`, right-click = context menu (switch pack / toggle eyes / hide / quit). Optional click-through mode (config) for users who want a pure ornament; in click-through mode activation is wake-word only.

**Failure containment.** Every Qt slot in `avatar/` wraps its body in `try/except` → `log + AvatarStatus(error=...)`; the NFR4 3-strikes rule disables the widget for the session. The pipeline must run identically with `avatar.enabled: false` — CI runs the full integration suite headless with the avatar off, and the avatar's own tests run separately (§19).

## 14. Security

Layered model unchanged. Avatar-specific analysis:

- The avatar adds **one** input path (`AVATAR_ACTIVATE`) which is equivalent in power to saying the wake word — it cannot trigger tools, only listening. Threat delta  $\approx 0$ .
- P10: pack loading treats every manifest field as data. Numeric fields are range-clamped, paths are resolved and **must remain inside the pack directory** (reject `../` traversal), image files are size-capped ( $\leq 2048 \times 2048$ ,  $\leq 5$  MB each, total pack  $\leq 40$  MB) and decoded via Qt's image loader only. No field is ever formatted into a shell command, file path outside the pack, or prompt.
- `set_avatar` is intentionally non-destructive/non-confirmed: worst case is a cosmetic change, and the arg is validated against a closed set (whitelist-by-construction, P2-consistent).
- Prompt-injection stance unchanged; note the avatar never touches transcripts or screen content (P9), so it is outside that attack surface entirely.

## 15. Configuration

Single frozen `config.yaml` as before. New section:

yaml

```
avatar:
  enabled: true
  packs_dir: "avatars/"
  active_pack: "cat"           # startup default; runtime switches persist to avatar_state.json
  scale: 1.0                   # 0.5-2.0
  eye_tracking: true
  eye_poll_hz: 30             # cap 30
  fps_cap: 30
  click_to_activate: true
  click_through: false
  blink: {min_s: 3, max_s: 8}
  fidget: {min_s: 20, max_s: 60}
```

**Frozen-config exception, justified:** pack switching at runtime (FR15) does not violate the "restart to change" rule's *reason* — that rule exists because hot-reloading model paths and whitelists is a security/correctness hazard. Pack identity is validated against a closed on-disk set, is cosmetic, and P10 keeps packs non-executable, so hot-swap carries none of the hazard the rule targets. `active_pack` in config is therefore only the startup default; the runtime value lives in `avatar_state.json` (§12). Everything else in the avatar section remains frozen. (Decision Log DL-8.)

## 16. Error Handling

Taxonomy unchanged. Avatar mappings: bad manifest / missing files → `PermanentError` for that pack (spoken notice, previous pack retained); Qt paint exceptions → contained per §13A with 3-strikes disable; pack directory disappearing mid-session → fall back to built-in minimal "dot" avatar (a bundled, code-drawn pulse circle that needs no assets — guarantees the state-visibility function survives any pack failure).

## 17. Logging

Two streams unchanged. Avatar events (pack switches, widget restarts, eye-tracking degradation) go to the **app log** with the current `turn_id` when triggered by a turn. Explicit prohibition: cursor coordinates and window position are never logged at any level (NFR1).

## 18. Coding Standards

Unchanged, plus: Qt imports confined to `avatar/` (§9); Qt slots follow the try/except containment pattern (§13A); the animator's state→animation mapping is a pure function (unit-testable without Qt); no `QApplication.processEvents()` calls anywhere (they re-enter the loop and break qasync's guarantees).

## 19. Testing Strategy

v1.0 suites unchanged, plus:

- **Unit (headless, no Qt):** manifest schema validation matrix (valid, traversal path, oversize image, missing state, unknown fields); animator state→animation mapping incl. fallback chain; eye-vector clamping and lerp math with synthetic cursor positions.
- **Integration:** `(pytest-qt)` offscreen (`(QT_QPA_PLATFORM=offscreen)`): pack load → state event → correct frame selected; hot-swap under a running frame timer; 3-strikes disable path; `(AVATAR_ACTIVATE)` ignored outside IDLE.
- **Latency CI addition:** the scripted turn from v1.0 now also asserts SC8 (event→frame-selection < 100 ms) and asserts the full pipeline latency budgets *with the avatar running*, catching qasync loop starvation early (the §7 escape-hatch trigger).
- **Manual smoke additions:** drag persistence across restart, right-click menu on each OS, click-through toggle, Wayland eye degradation notice.

## 20. Development Workflow & Milestones

M1-M7 unchanged. New milestone, deliberately after the assistant works end-to-end (the avatar is a view; there must be something to view):

- **M8 — Face:** avatar package + `(avatars/cat/)` reference pack; window, animator, eye tracking, click-to-activate, context menu, `(set_avatar)` tool, hot-swap, fallback dot avatar; SC8-SC10 measured; latency CI extended; Appendix B validated by building the `(axolotl)` pack purely from its instructions (the doc test: if the second pack needs any code change, Appendix B or `(packs.py)` is wrong).
- **M7 note:** the handbook v1.1→v1.2 as-built update now happens after M8.

## 21. Deployment

Unchanged, plus dependency note: PySide6 + qasync enter `(pyproject.toml)`; on Linux, document the Qt xcb/wayland runtime packages in `(models.md)`'s sibling `(setup.md)`. The `systemd --user` unit needs a graphical-session target dependency once the avatar exists (`(After=graphical-session.target)`).

## 22. Latency & Resource Budgets (binding targets)

| Span                                         | Tier A (CPU)       | Tier B (GPU) | Mechanism                                      |
|----------------------------------------------|--------------------|--------------|------------------------------------------------|
| Wake detection                               | <150 ms            | <100 ms      | frame-level ONNX                               |
| End-of-utterance                             | ~700 ms<br>silence | same         | Silero trailing window                         |
| Speech-end → first token                     | <1500 ms           | <600 ms      | warm Ollama, streamed                          |
| First token → first audio                    | <800 ms            | <400 ms      | sentence-1 chunk → Piper                       |
| Barge-in stop                                | <100 ms            | <100 ms      | CANCEL + queue flush                           |
| State event → avatar frame<br>( <i>new</i> ) | <100 ms            | <100 ms      | bus fan-out + latest-wins mailbox              |
| Pack hot-swap ( <i>new</i> )                 | <1 s               | <1 s         | pre-validated manifest, lazy frame<br>decode   |
| Avatar idle CPU ( <i>new</i> )               | <3% of one<br>core | same         | timer-driven blits, static-frame timer<br>stop |

### 23. Known Tradeoffs (accepted deliberately)

T1-T6 unchanged. New:

- **T7** qasync couples GUI and pipeline into one loop; a pathological Qt paint could add jitter. Accepted because paints are micro-blits, it's continuously measured by latency CI, and P9 makes the avatar the cheapest component to evict to its own process if the trigger fires.
- **T8** Sprite-sheet 2D only — no rigged/skeletal animation (Live2D-style). Accepted: keeps packs pure data (P10), keeps CPU budget trivial, and matches the pixel-pet aesthetic. Revisit only via §28.
- **T9** Cursor polling at 30 Hz is a compromise between eye smoothness and idle CPU; eyes will visibly "step" under very fast cursor motion. Accepted; lerp smoothing hides most of it.
- **T10** On Wayland, global cursor tracking degrades (§13A). Accepted; platform limitation, spoken/logged once.

### 24. Decision Log (format + seed entries)

DL-1 ... DL-6 unchanged. New:

- **DL-7** | v1.1 | Avatar rendered with PySide6 in-process via qasync | single loop, zero IPC (extends DL-1); Qt gives transparency/always-on-top/tray portably | *rejected*: separate Electron/web overlay (heavyweight, IPC), pygame window (poor transparency/always-on-top support cross-platform), separate Python process (IPC + duplicated state).
- **DL-8** | v1.1 | Runtime avatar-pack hot-swap as the sole frozen-config exception | cosmetic, closed-set-validated, non-executable data (P10) — none of the hazards the frozen rule targets | *rejected*: restart-to-switch (kills the delight of the feature), general config hot-reload (the actual hazard).
- **DL-9** | v1.1 | Avatar packs = declarative YAML + images, no code | P10; casual pack sharing must not be a code-execution vector | *rejected*: Python plugin packs (flexible but unacceptable on a machine with a tool executor).
- **DL-10** | v1.1 | Click-to-activate  $\equiv$  wake word; click never barge-ins | keeps interrupt semantics single-sourced in D-5's voice analysis; clicking a speaking avatar is too easy to do accidentally while dragging | *rejected*: click = universal interrupt.

## 25. Open Questions (blocking or soon-blocking)

D-1 ... D-5 unchanged. New:

- **D-6**: Which two launch packs, and their art source? Owner named a cat and an axolotl. Options: (a) owner-provided/commissioned sprites, (b) CC0/CC-BY pixel-art assets (license file required inside the pack — Appendix B), (c) AI-generated sprite sheets. Blocks M8 asset work only; code is art-agnostic. Default: build the fallback dot avatar + placeholder cat from simple geometric shapes so M8 code lands regardless.
- **D-7**: Should the avatar show *listening feedback* beyond the state animation — e.g., ears/gills twitch with mic amplitude? Requires a low-rate RMS side-channel from audio-in (still P6-safe: computed outside the callback). Delightful but adds a queue; default: defer to §28.
- **D-8**: Multi-monitor behavior — pin to one screen, or remember per-screen positions? Default: single remembered position, clamped into visible geometry at startup.

## 26. Repository Structure

As §9, adding root-level `avatars/` (packs, each with its own `LICENSE/``CREDITS` file), `avatar_state.json` (gitignored), and `setup.md` (Qt runtime deps). `.github/workflows/ci.yaml` gains the `QT_QPA_PLATFORM=offscreen` avatar test job.

## 27. Service-Split Triggers

Unchanged, with the §7 note formalized: (d) latency CI attributes budget violations to Qt paint jitter → avatar moves to its own process behind a one-way event socket (its P9 interface makes this a mechanical extraction).

## 28. Future Extensions (ordered by owner interest)

1. GUI control (act on screen) — unchanged.
2. Cloud fallback brain — unchanged.
3. Cross-session memory — unchanged.
4. Hindi / code-switched responses — unchanged.
5. Custom cloned voice — unchanged.
6. *(new)* **Avatar life+**: mic-amplitude ear/gill twitch (D-7), speech-viseme mouth flaps, dragging physics, edge-walking, reactions to tool results (confetti on success, sweat-drop on error), seasonal outfits via overlay layers.
7. *(new)* **Community pack format v2**: rigged animation (Live2D/Spine-style) — requires revisiting T8/P10 with a sandboxed runtime; do not attempt casually.

## 29. Handbook Maintenance Rules

Unchanged. Reminder: this v1.1 bump was mandated by the §10 interface additions and new §3 principles (P9/P10).

---

## Appendix A — Claude Code Kickoff Prompts

A.1 and A.2 unchanged from v1.0 (they reference milestones generically; M8 is picked up automatically by A.2's "first incomplete milestone" logic). One line appended to A.1's task framing: *"Milestones now run M1–M8; the avatar subsystem (M8, §13A, Appendix B) is last and must not be started before M7 passes."*

## Appendix B — Avatar Pack Authoring Guide *(new)*

This is the complete recipe for adding a new avatar. **If a step here requires touching Python, the pack format is broken — file it as a bug, not a workaround** (M8's doc test).

### B.1 Pack anatomy

```

avatars/
└─ axolotl/                                # pack name = directory name (lowercase, [a-
z0-9_-])
    ├─ avatar.yaml                          # manifest (required)
    ├─ LICENSE                             # art license/credits (required)
    ├─ idle/                               # one directory per animation, numbered frames
    │   ├─ 000.png
    │   ├─ 001.png
    │   └─ ...
    ├─ listening/
    ├─ thinking/
    ├─ speaking/
    ├─ error/                             # optional
    ├─ blink/                             # optional overlay
    ├─ fidget_01/                          # optional, any number of fidgets
    └─ eyes/
        └─ pupil.png                      # optional; enables eye tracking

```

All frames within one pack share identical pixel dimensions. PNG with alpha. Per-image  $\leq 2048 \times 2048$  and  $\leq 5$  MB; whole pack  $\leq 40$  MB (enforced by `packs.py`).

## B.2 Manifest (`avatar.yaml`)

yaml

```
name: "Axolotl"           # display name for menus/voice
version: 1
author: "owner"
frame_size: [128, 128]    # must match every frame
anchor: bottom-center     # how the window hugs its position

states:                   # orchestrator state → animation directory
  idle: {anim: idle,      fps: 8,  loop: true}
  listening: {anim: listening, fps: 12, loop: true}
  thinking: {anim: thinking, fps: 10, loop: true}
  speaking: {anim: speaking, fps: 12, loop: true}
  error: {anim: error,    fps: 6,  loop: false} # holds last frame

fallbacks:                # used when a states entry or directory is missing
  speaking: idle
  thinking: idle
  listening: idle
  error: idle

overlays:
  blink: {anim: blink, fps: 16, only_in: [idle]}
  fidgets: [fidget_01]    # picked randomly on the fidget timer

eyes:                     # omit this whole block for no eye tracking
  pupil: eyes/pupil.png   # path inside the pack only
  sockets:                 # one entry per eye, coords in frame pixels
    - {center: [46, 52], radius: 6}
    - {center: [82, 52], radius: 6}
  hide_in: [blink, error] # states/overlays where pupils are not drawn
```

Validation rules (enforced, not advisory): unknown top-level keys rejected; every path must resolve inside the pack directory; `fps` clamped to 1–30; every `states.*.anim` and `fallbacks.*` target must exist as a frame directory; socket centers must lie within `frame_size`.

### B.3 Step-by-step: adding a new avatar

1. Copy an existing pack directory (e.g., `avatars/cat/`) to `avatars/<yourname>/`.
2. Replace the frame PNGs. Minimum viable pack: an `idle/` directory with one frame and a manifest whose `fallbacks` route every state to `idle`. Everything else is progressive enhancement.
3. Edit `avatar.yaml`: set `name`, `frame_size`, and the eye socket coordinates (open a frame in any image editor, note the pixel centers of the eyes and a pupil-travel radius; omit the `eyes:` block if the character shouldn't track the cursor).
4. Add the art's `LICENSE`/credits file. Non-negotiable even for self-made art (future-you will forget provenance).
5. Restart the assistant *or* say "rescan avatars" / use the tray menu's rescan — the pack is auto-discovered and joins the switch list, the voice glossary, and `set_avatar`'s valid set.
6. Say "switch to <name>" or pick it from the right-click menu. If the manifest is invalid, the assistant speaks the first validation error and keeps the current pack.

## B.4 Design tips for packs that feel alive

- Idle should breathe: a 2-4 frame subtle loop at low fps beats a static image.
- Listening should be instantly distinct from idle (ears up / gills flared) — this animation is the SC8 feedback the user relies on.
- Thinking benefits from an obvious loop (tail flick, floating bubbles) since THINKING is the state users stare at.
- Keep pupils small relative to socket radius; large pupils with small travel look dead.
- Error should be endearing, not alarming — this is a companion, and errors are spoken anyway.

*End of Handbook v1.1. The repo copy becomes authoritative upon first commit.*